

NTT 多項式乘法：從直覺到加速

一、問題：多項式乘法很慢

把兩個多項式相乘，例如

$$(1 + 2x) \times (5 + 6x) = 5 + 16x + 12x^2$$

做法是「每一項都要跟對方每一項相乘」。若兩個多項式各有 n 項，就要做 $n \times n$ 次乘法，複雜度 $O(n^2)$ 。當 $n = 1024$ 時就是上百萬次乘法，太慢。

二、關鍵點子：換一種「表示法」

一個多項式有兩種等價的表示方式（以 $f(x) = 5 + 16x + 12x^2$ 為例）：

表示法	內容	例子
係數表示	列出係數	[5, 16, 12]
點值表示	代幾個 x 進去記錄結果	$f(0) = 5, f(1) = 33, f(2) = 81, \dots$

重點：用點值表示時，多項式相乘超級簡單——只要把對應點的值相乘就好。

因為若 $h(x) = f(x) \cdot g(x)$ ，則在任何一點 x_0 都有 $h(x_0) = f(x_0) \cdot g(x_0)$ 。一個點只要 1 次乘法， n 個點只需 n 次，這一步是 $O(n)$ ，遠快於 $O(n^2)$ 。

三、整個流程變成三步

係數	--(1)求值-->	點值	(f, g 各算出一堆點的值)
點值	--(2)逐點相乘-->	點值	(便宜, $O(n)$)
點值	--(3)插值-->	係數	(把答案的點值變回係數)

問題只剩：步驟 (1) 求值、(3) 插值本身會不會也很慢？若隨便挑點，求值一樣是 $O(n^2)$ ，等於沒賺到。

四、FFT / NTT 的魔法：挑「特別的點」

若代入的點不是隨便挑，而是挑**單位根**——一組具對稱循環性質的特殊數字——那麼求值與插值可用「分而治之」加速到 $O(n \log n)$ 。

- **FFT**：用複數單位根 $e^{2\pi i/n}$ (含小數，有浮點誤差)。
- **NTT**：用「模 q 整數版」的單位根 (全程整數、完全精確)。

NTT 就是把 FFT 搬到「整數 + 取餘數 (mod q)」的世界，讓密碼學需要的整數運算不會有誤差。單位根存在的條件是 q 為質數且 $q \equiv 1 \pmod{n}$ (負循環卷積則需 $q \equiv 1 \pmod{2n}$)。

五、一句話總結

NTT 把「慢的係數相乘」先轉成「快的點值相乘」，靠特殊的單位根讓來回轉換也很快，整體從 $O(n^2)$ 降到 $O(n \log n)$ 。

六、與 Ring-LWE 的關聯

Ring-LWE 在環 $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ 上運算，乘法是**負循環卷積**：超出 x^n 的項因 $x^n \equiv -1$ 要變號折回。作法是改用 $2n$ 次單位根 ψ ($\psi^2 = \omega$)，對係數先乘權重 ψ^i 、做完 NTT 逐點相乘後再乘 ψ^{-i} ，即可把負循環卷積轉成一般逐點相乘。這正是 Ring-LWE 相較 Standard LWE 的核心效能突破，也是 Kyber、Dilithium 等 NIST 後量子標準能高速運行的基礎。

七、實際執行範例

以下為 Python 實作 (NTT 法 vs. 樸素 $O(n^2)$ 法對拍) 的實際輸出。兩法結果完全一致，驗證 NTT 正確：

```
模數 q = 17 (質數: True) · 維度 n = 8
原根 g = 3
a(x) = 1 + 2x^1 + 3x^2 + 4x^3
b(x) = 5 + 6x^1 + 7x^2 + 8x^3
-----
[循環卷積 Z_q[x]/(x^n - 1)]
NTT : [5, 16, 0, 9, 10, 1, 15, 0]
樸素 : [5, 16, 0, 9, 10, 1, 15, 0]
一致 : True
-----
[負循環卷積 Z_q[x]/(x^n + 1) <- Ring-LWE]
NTT : [5, 16, 0, 9, 10, 1, 15, 0]
樸素 : [5, 16, 0, 9, 10, 1, 15, 0]
一致 : True
-----
隨機測試 n=256: 負循環卷積 NTT == 樸素 -> True
```

讀法說明：

- $a(x) = 1 + 2x + 3x^2 + 4x^3$ 、 $b(x) = 5 + 6x + 7x^2 + 8x^3$ ，係數由低次到高次列出。
- **模數選最小可行的** $q = 17$ ：需 q 為質數且 $q \equiv 1 \pmod{2n} = 16$ ，最小者即 17。普通整數乘積為 $[5, 16, 34, 60, 61, 52, 32]$ ，在 $\text{mod} 17$ 下真正被折回——例如 $34 \equiv 0$ 、 $60 \equiv 9$ 、 $61 \equiv 10$ 、 $52 \equiv 1$ 、 $32 \equiv 15$ ，因此能清楚看到模運算的作用。(若改用大模數如 7681，最大係數 61 遠小於模數，取餘數不會發生，輸出就跟普通整數乘法無異，看不出 $\text{mod} q$ 的意義。)
- 此例乘積最高次為 x^6 ，未達 x^8 ，故未觸發 $x^n \equiv -1$ 的折回，循環與負循環結果恰好相同。
- 真正的負循環折回由最後 $n = 256$ 的隨機測試驗證 (該測試用 $q = 7681$ ，因需 $q \equiv 1 \pmod{512}$)：在大規模隨機係數下，NTT 與樸素法仍完全一致。